**popkovden** 15 мар в 01:25

# Drag and Drop в Jetpack Compose

Простой 5 мин 1.2K

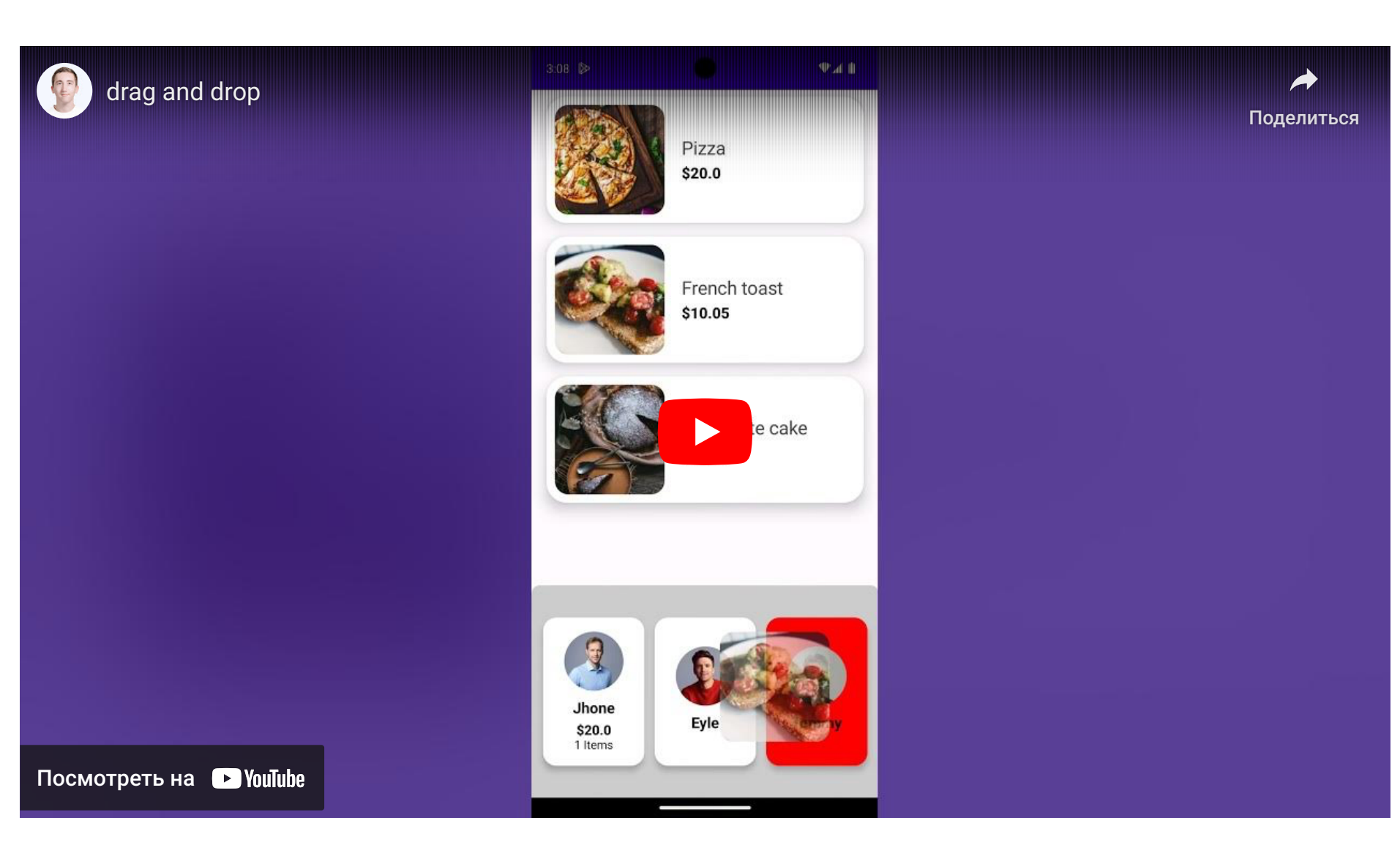
Разработка под Android*, Jetpack Compose*

Туториал Перевод

Автор оригинала: Radhika Saliya

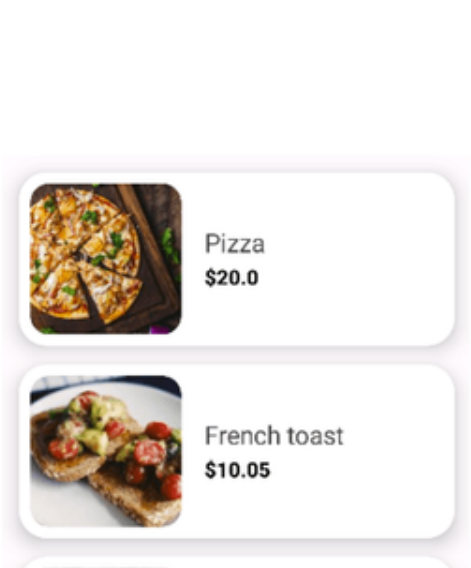
## Вступление

В январе 2024, крупное обновление Jetpack Compose добавило два новых модификатора: `dragAndDropSource` и `dragAndDropTarget`. В этой статье я расскажу как реализовать эффект Drag and Drop в Jetpack Compose.



## Composable функция для отображения карточки еды

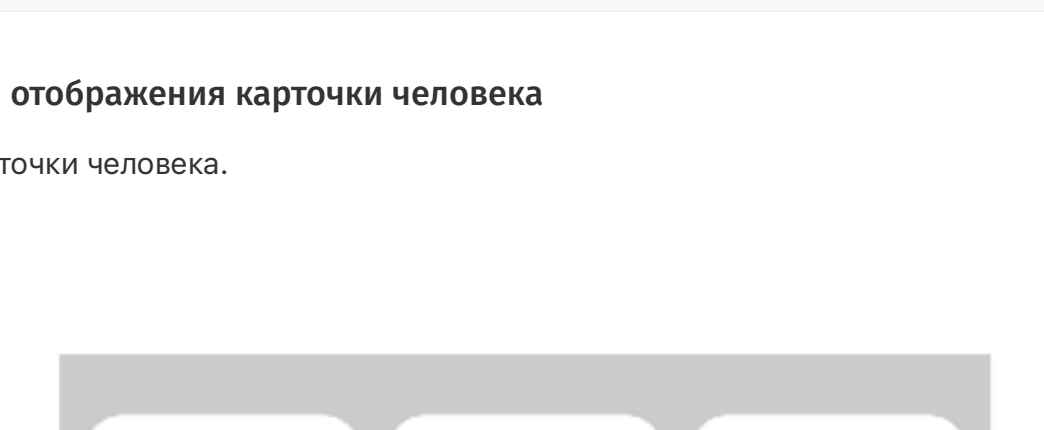
Сперва напишем код для карточки еды.



```
fun FoodItemCard(foodItem: FoodItem) {
    Card(
        elevation = CardDefaults.elevatedCardElevation(defaultElevation = 10.dp),
        colors = CardDefaults.elevatedCardColors(
            containerColor = Color.White,
        ), shape = RoundedCornerShape(24.dp),
        modifier = Modifier.padding(8.dp)
    ) {
        Row(
            verticalAlignment = Alignment.CenterVertically,
            modifier = Modifier.padding(10.dp)
        ) {
            Image(
                painter = painterResource(id = foodItem.image),
                contentDescription = null,
                contentScale = ContentScale.Crop,
                modifier = Modifier
                    .size(130.dp)
                    .clip(RoundedCornerShape(16.dp))

            )

            Spacer(modifier = Modifier.width(20.dp))
            Column(modifier = Modifier.weight(1f)) {
                Text(
                    text = foodItem.name,
                    fontSize = 22.sp,
                    color = Color.DarkGray
                )
                Spacer(modifier = Modifier.height(6.dp))
                Text(
                    text = "$${foodItem.price}",
                    fontSize = 18.sp,
                    color = Color.Black,
                    fontWeight = FontWeight.ExtraBold
                )
            }
        }
    }
}

LazyColumn(
    modifier = Modifier.fillMaxSize(),
    contentPadding = PaddingValues(horizontal = 10.dp)
) {
    items(items = foodList) { food ->
        FoodItemCard(foodItem = food)
    }
}
```

## Composable функция для отображения карточки человека

Далее напишем код для карточки человека.



```
@Composable
fun PersonCard(person: Person) {
        Column(
            modifier = Modifier
                .padding(6.dp)
                .shadow(elevation = 4.dp, shape = RoundedCornerShape(16.dp))
                .width(width = 120.dp)
                .fillMaxHeight(0.8f)
                .background(Color.White, RoundedCornerShape(16.dp)),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Image(
                painter = painterResource(id = person.profile), contentDescription = null,
                modifier = Modifier
                    .size(70.dp)
                    .clip(CircleShape),
                contentScale = ContentScale.Crop
            )
            Spacer(modifier = Modifier.height(10.dp))
            Text(
                text = person.name,
                fontSize = 18.sp,
                color = Color.Black,
                fontWeight = FontWeight.Bold
            )
        }
}

LazyRow(
    modifier = Modifier
        .fillMaxHeight(0.3f)
        .fillMaxWidth()
        .background(Color.LightGray, shape = RoundedCornerShape(topEnd = 10.dp, topStart = 10.dp))
        .padding(vertical = 10.dp)
        .align(Alignment.BottomCenter),
    verticalAlignment = Alignment.CenterVertically,
    horizontalArrangement = Arrangement.Center
) {
    items(items = persons) { person ->
        PersonCard(person)
    }
}
```

## Добавим источник для Drag and Drop

Перед тем как начнём, небольшая справка по модификаторам:

**Modifier.dragAndDropSource**

Этот модификатор позволяет компоненту стать источником Drag and Drop.

```
@Composable
fun MyDraggableComponent() {
    Box(
        modifier = Modifier
            .size(100.dp)
            .background(Color.Blue)
            .dragAndDropSource(
                drawDragDecoration = {
                    // UI перетаскиваемого элемента
                }
            ) {
                // логика обработки Drag and Drop
                startTransfer (/* данные для передачи */ )
            }
    ) { /* контент перетаскиваемого компонента */ }
}
```

- `Modifier.dragAndDropSource` принимает два параметра: `drawDragDecoration` и `block`;
- `drawDragDecoration` — это лямбда блок, который предоставляет визуальное представление компонента, который перетаскивается Drag and Drop'ом;
- `block` это лямбда блок, он принимает `DragAndDropSourceScope` как ресивер. Он позволяет обнаружить Drag and Drop жест и в последствии вы можете его обработать;
- вызов `startTransfer` в лямбде инициализирует Drag and Drop действие.

Теперь давайте добавим этот модификатор на карточку еды:

Тут, `transferData` определяет контент, который должен передаваться при Drag and Drop. Например, передача данных из карточки еды в карточку человека.

```
private const val foodItemTransferAction = "action_foodItem"
private const val foodItemTransferData = "data_foofdItem"
...
startTransfer(
    DragAndDropTransferData(
        clipData = ClipData.newIntent(
            "foodItem",
            Intent(foodItemTransferAction).apply {
                putExtra(
                    foodItemTransferData,
                    Gson().toJson(foodItem)
                )
            },
        )
    )
```

Теперь карточка еды стала перетаскиваемой.

## Добавим место для Drop and Drop

Вторая справка по модификаторам:

**Modifier.dragAndDropTarget**

Этот модификатор в Jetpack Compose позволяет composable функциям получить события от Drop and Drop.

```
@Composable
fun MyDragTarget() {
    Box(
        modifier = Modifier
            .size(100.dp)
            .background(Color.Green)
            .dragAndDropTarget(
                shouldStartDragAndDrop = { startEvent-> return true },
                target =  object : DragAndDropTarget { ... }
            )
    ) { /* контент перетаскиваемого компонента */ }
}
```

Он принимает два параметра:

1. `shouldStartDragAndDrop`: говорит компоненту нужно ли получать события перетаскивания от `DragAndDropEvent`.
2. `target` — `DragAndDropTarget` получает следующие события от Drag and Drop:
    1. `onDrop(event)`: эта функция вызовется, когда компонент будет перетащен внутрь `DragAndDropTarget`. Значение `true` указывает на то, что событие от `DragAndDropEvent` было обработано, напротив `false` указывает на то, что оно было отклонено;
    2. `onStarted(event)`: эта функция вызывается, когда происходит Drag and Drop жест. Позволяет установить состояние для `DragAndDropTarget` во время подготовки к этой операции;
    3. `onEntered(event)`, `onMoved(event)`, `onExited(event)`: эти функции, вызываются когда элемент находится/двигается в области `DragAndDropTarget`;
    4. `onChanged(event)`: эта функция вызывается, когда событие Drag and Drop меняется в `DragAndDropTarget` области;
    5. `onEnded(event)`: эта функция вызывается, когда событие Drag and Drop завершено. Все экземпляры `DragAndDropTarget` в иерархии, которые ранее получили событие `onStarted` получат это событие, что позволяет сбросить состояние для `DragAndDropTarget`.

В нашем примере PersonCard — это цель Drop, поэтому давайте добавим этот модификатор карточке PersonCard.

```
fun PersonCard(person: Person) {
    // состояние для связки элемента еды и человека
    val foodItems = remember { mutableStateMapOf<Int, FoodItem>() }
    Column(
        modifier = Modifier
             ....
            .background(Color.White, RoundedCornerShape(16.dp))
            .dragAndDropTarget(
                shouldStartDragAndDrop = { event ->
                    // проверяет, если Drag and Drop содержит текст intent mime типа
                    event.mimeTypes().contains(ClipDescription.MIMETYPE_TEXT_INTENT)
                },
                target = object : DragAndDropTarget {
                    override fun onDrop(event: DragAndDropEvent): Boolean {
                        // достает элемент еды из Drag and Drop события и добавляет его в состояние
                        val foodItem = event.toAndroidDragEvent().clipData.foodItem() ?: return false
                        foodItems[foodItem.id] = foodItem
                        return true
                    }
                }
            ),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) { // контент карточки PersonCard }

private fun ClipData.foodItem(): FoodItem? {
    return (0 until itemCount)
        .mapNotNull(::getItemAt).firstNotNullOfOrNull { item ->
            item.intent?.getStringExtra(foodItemTransferData)?.takeIf { it.isNotEmpty() }
        }?.let { Gson().fromJson(it, FoodItem::class.java) }
}
```

Внутри `onDrop` функции, мы извлекаем элемент еды Drag and Drop события и добавляем `foodItems` состояние.

Теперь, чтобы цель Drop изменила свой цвет, когда источник находится в области желаемого Drag and Drop. Нужно прослушать события `onEntered` и `onExited`.

```
@OptIn(ExperimentalFoundationApi::class)
@Composable
fun PersonCard(person: Person) {
   var bgColor by remember { mutableStateOf(Color.White) }

    Column(
        modifier = Modifier
            ....
            .background(bgColor, RoundedCornerShape(16.dp))
            .dragAndDropTarget(
                shouldStartDragAndDrop = { event -> ... },
                target = object : DragAndDropTarget {
                    override fun onDrop(event: DragAndDropEvent): Boolean {
                        ...
                        bgColor = Color.White
                        return true
                    }

                    override fun onEntered(event: DragAndDropEvent) {
                        super.onEntered(event)
                        bgColor = Color.Red
                    }

                    override fun onExited(event: DragAndDropEvent) {
                        super.onExited(event)
                        bgColor = Color.White
                    }


                }
            ),
    ) { /* карточка PersonCard */ }
```

> Исходный код из статьи вы можете найти в Github репозитории — https://github.com/cp-radhika-s/Drag_and_drop_jetpack_compose/tree/explore-modifier-drag-drop-source

**Теги:** jetpack compose, drag-and-drop

**Хабы:** Разработка под Android, Jetpack Compose

+2 13 0

**Редакторский дайджест**

Присылаем лучшие статьи раз в месяц

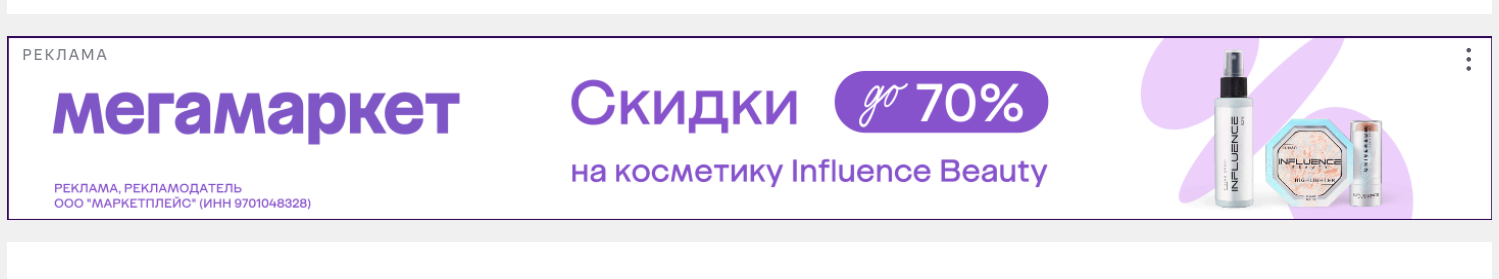
Электропочта

8 Карма 19.8 Рейтинг

**Денис** @popkovden

Android-разработчик в команде Лайв Тайпинг

Подписаться


РЕКЛАМА
мегамаркет
РЕКЛАМА. РЕКЛАМОДАТЕЛЬ ООО "МАРКЕТПЛЕЙС" (ИНН 9701048328)
Скидки до 70%
на косметику Influence Beauty

**Комментарии**

Здесь пока нет ни одного комментария, вы можете стать первым!

Зарегистрируйтесь на Хабре, чтобы оставить комментарий

## Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ

**JustJeremy** 6 часов назад

**Если покупка — это не владение, то пиратство — не воровство**

6 мин 8.7K

Мнение Перевод

+72 35 47 +47

**Meklon** 12 часов назад

**Как я собрал красивое ведро для гидропоники**

Простой 10 мин 7.4K

Туториал

+72 77 101 +101

**dlinyj** 9 часов назад

**Тюнинг старого тетриса**

Простой 4 мин 3.8K

Туториал

+43 16 35 +35

**BabayMazay** 8 часов назад

**Несколько самодельных стеклодувных приспособлений и инструментов. Текущее дооснащение домашней мастерской**

Простой 8 мин 887

Туториал

+36 9 5 +5

**slavanikolsky** 20 часов назад

**Сделано в СССР. Советский истребитель спутников — первым в мире перехватил космический аппарат-мишень**

Простой 10 мин 21K

+27 21 69 +69

**it_union** 11 часов назад

**Увольнение без компенсаций: как этого избежать и что вам положено по закону**

9 мин 4.9K

+22 27 3 +3

**Maksvelis** 5 часов назад

**Выжимаем максимум из PostgreSQL**

Средний 14 мин 1.9K

Обзор

+21 41 0

**olegantipovDDG** 9 часов назад

**Роботы-покорители Венеры: 40 лет забытому подвигу советской науки**

Простой 8 мин 2.1K

+21 11 6 +6

**Antxak** 8 часов назад

**Мошеннические схемы, так ли их много?**

Простой 5 мин 1.7K

Мнение

+17 14 4 +4

**ancotir** 6 часов назад

**Палочки Непера и «Шпаргалка» — карманные «калькуляторы» XVII века для умножения и деления сложных чисел**

Простой 10 мин 751

Ретроспектива

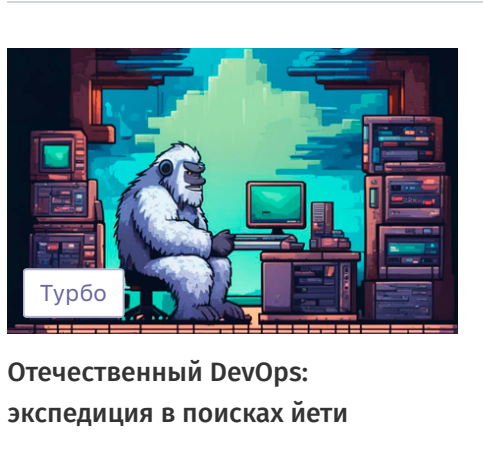
+15 10 1 +1

**Проблемы со связью и боязнь облаков: а при чём тут DevOps?**

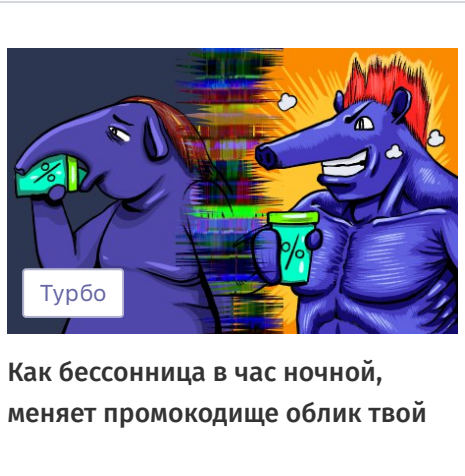
Турбо

Показать еще

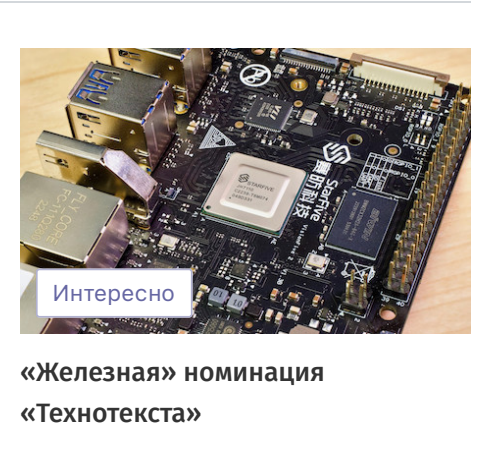
МИНУТОЧКУ ВНИМАНИЯ



Отечественный DevOps: экспедиция в поисках йети



Как бессонница в час ночной, меняет промокодище облик твой

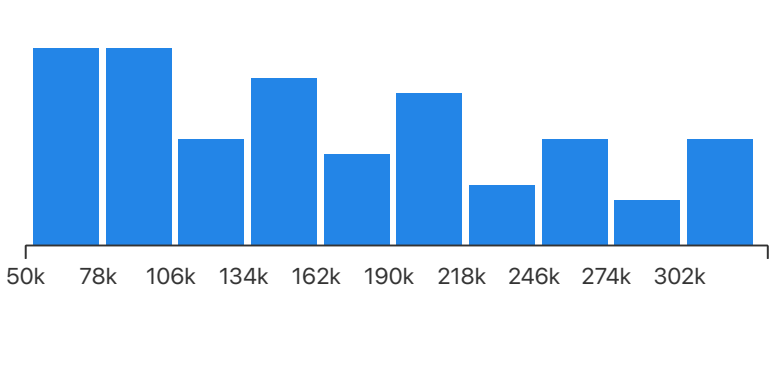


«Железная» номинация «Технотекста»

СРЕДНЯЯ ЗАРПЛАТА В IT

**167 692 ₽/мес.**

— средняя зарплата во всех IT-специализациях по данным из 28 570 анкет, за 1-ое пол. 2024 года. Проверьте «в рынке» ли ваша зарплата или нет!



Проверить свою зарплату